Market Data

1 Overview

Pricing an option requires two distinct types of input. The first is the contract specification — the strike, expiry, and option type defined in `EuropeanOption`. The second is a snapshot of observable market conditions at the moment of pricing: where is the underlying trading, what are interest rates, what is the dividend yield? These inputs live in `MarketData`.

Note that **volatility is not in `MarketData`**. Volatility is a parameter of the stochastic model assumed to drive the underlying — it belongs on the model (e.g. `BlackScholesModel`), not on the market. This distinction becomes important as the library expands: the Heston model has multiple volatility-related parameters, and none of them are "market observables" in the same sense that spot price or interest rates are.

The separation matters: the contract is fixed once written, while market conditions change continuously. Holding the contract and model constant while varying `MarketData` is how you run scenario analyses and stress tests.

2 The Inputs

2.1 Spot Price (S)

The **spot price** is the current market price of the underlying asset — the price at which you could buy or sell it right now. It is the most direct input to the option price: a call becomes more valuable as the underlying rises (you are closer to, or further into, the money), and a put becomes more valuable as it falls.

Spot is always strictly positive; a negative price is not economically meaningful.

2.2 Risk-Free Rate (r)

The **risk-free rate** represents the return available on a riskless investment over the option's life. In practice, this is typically proxied by:

- Government bond yields (e.g. US Treasuries, UK Gilts) for longer-dated options
- Overnight index swap (OIS) rates or SOFR for shorter tenors

In the Black-Scholes framework, r is expressed as a **continuously compounded** annual rate. Continuous compounding might feel unfamiliar if you are used to annual or quarterly rates, but it simplifies the mathematics considerably. The two are related by:

$$r_{\text{cont}} = \ln(1 + r_{\text{annual}})$$

So a 5% annually compounded rate corresponds to $r_{\text{cont}} = \ln(1.05) \approx 4.879\%$.

The risk-free rate enters option pricing via the **discounting** of future cash flows: a payoff received at time T in the future is worth e^{-rT} today. It also captures the *cost of carry* — owning the underlying instead of a risk-free bond has an opportunity cost, which matters when constructing the replicating portfolio underlying the Black-Scholes argument.

The rate can be negative (as has been observed in various economies post-2008 and post-COVID), which the model handles without issue.

2.3 Dividend Yield (q)

The **continuous dividend yield** accounts for income paid by the underlying asset during the option's life. For equities, dividends reduce the stock price on the ex-dividend date — a call holder misses out on this income (since they do not own the stock), while a put holder benefits.

In the Merton (1973) extension of Black-Scholes, dividends are modelled as a **continuous yield** q: the stock is assumed to continuously bleed value at rate q. The effective forward price of the stock becomes $Se^{(r-q)T}$ rather than Se^{rT}. This substitution flows through the entire pricing formula.

For many practical applications — liquid equities, indices, and forex (where q represents the foreign interest rate) — the continuous yield is a reasonable approximation. For individual stocks paying lumpy quarterly dividends, more sophisticated adjustments are needed, but that is beyond the scope of this model.

The default is $q = 0$, which is appropriate for non-dividend-paying assets. It can also take negative values in principle, though this is economically unusual.

3 The `MarketData` Class

```python
from options_pricer import MarketData

# Non-dividend-paying asset: spot=100, rate=5%
md = MarketData(spot=100.0, rate=0.05)

# With a 2% continuous dividend yield
md_div = MarketData(spot=100.0, rate=0.05, div_yield=0.02)

# Negative rates are valid
md_neg_rate = MarketData(spot=100.0, rate=-0.005)
```

Invalid inputs raise `ValueError` at construction, not at pricing time:

```python
# These will raise ValueError
MarketData(spot=0.0,   rate=0.05)   # spot must be positive
MarketData(spot=-50.0, rate=0.05)   # spot must be positive
```

4 Using `MarketData` for Scenario Analysis

Because `MarketData` is a plain dataclass, running scenarios is straightforward. Use `dataclasses.replace()` to create variants with one field changed:

```python
from options_pricer import (
    BlackScholesModel, ClosedFormPricer, EuropeanOption, MarketData, OptionType,
)
import dataclasses

contract = EuropeanOption(option_type=OptionType.CALL, strike=100.0, expiry=1.0)
model    = BlackScholesModel(vol=0.20)
base     = MarketData(spot=100.0, rate=0.05)

# Price across a range of spot levels
for spot in [80, 90, 95, 100, 105, 110, 120]:
    md = dataclasses.replace(base, spot=float(spot))
    print(f"S={spot:3d}  price={ClosedFormPricer.price(contract, md, model):.4f}")
```

`dataclasses.replace()` creates a new `MarketData` with one field changed, leaving the rest intact — a clean pattern for sensitivity analysis without mutating any existing objects.